Build: Search Bar with Debounced API Calls

2 min read Updated 23 Apr 2026 Topic 52 of 62

Problem: Build a search input that calls an API after the user stops typing (debounced). Show loading and results.

What This Tests: Custom hooks (useDebounce), useEffect with dependencies, loading/error states, API integration.

jsx

JSX Copy

```jsx
function useDebounce(value, delay = 500) {
  const [debouncedValue, setDebouncedValue] = useState(value);

  useEffect(() => {
    const timer = setTimeout(() => setDebouncedValue(value), delay);
    return () => clearTimeout(timer);
  }, [value, delay]);

  return debouncedValue;
}

function SearchBar() {
  const [query, setQuery] = useState('');
  const [results, setResults] = useState([]);
  const [loading, setLoading] = useState(false);
  const debouncedQuery = useDebounce(query, 300);

  useEffect(() => {
    if (!debouncedQuery.trim()) {
      setResults([]);
      return;
    }
```

```
    setLoading(true);

    fetch(`https://api.github.com/search/users?q=${debouncedQuery}`)
      .then(res => res.json())
      .then(data => {
        if (!cancelled) {
          setResults(data.items || []);
          setLoading(false);
        }
      })
      .catch(() => {
        if (!cancelled) setLoading(false);
      });

    return () => { cancelled = true; };
  }, [debouncedQuery]);

  return (
    <div>
      <input value={query} onChange={e => setQuery(e.target.value)} placeh

      {loading && <p>Searching ... </p>}

      {!loading && results.length > 0 && (
        <ul>
          {results.slice(0, 10).map(user => (
            <li key={user.id}>
              <img src={user.avatar_url} alt={user.login} width={32} heigh
              {user.login}
            </li>
          ))}
        </ul>
      )}

      {!loading && debouncedQuery && results.length === 0 && <p>No results
    </div>
  );
}
```

Key Concepts: Custom useDebounce hook, race condition prevention with cancelled flag, conditional rendering for loading/empty/results states, limiting displayed results.

QUICK CHECK — no marks, just for you.

1. In the SearchBar component, what is the purpose of routing the input value through useDebounce before using it as the fetch dependency?

- ◯ So the input stays controlled while the request is in flight
- ◯ So the API is only called once the user stops typing, instead of on every keystroke
- ◯ So React batches the setQuery and setResults updates into one render
- ◯ So the results array is memoized and does not re-render the list

Check answer

2. Inside useDebounce, the effect returns a cleanup that calls clearTimeout(timer), and lists [value, delay] as dependencies. What happens when the user types a new character before the delay elapses?

- ◯ The new value is ignored until the first timer fires and updates debouncedValue
- ◯ Both timers stay alive, so debouncedValue updates twice in quick succession
- ◯ The cleanup only runs when the component unmounts, so the old timer still fires
- ◯ The pending timer is cleared and a fresh one starts, so debouncedValue never updates mid-typing

Check answer

3. The fetch effect declares `let cancelled = false;` and returns `() => { cancelled = true; }`, guarding every setState with `if (!cancelled)`. What problem does this solve?

- ○ A slower earlier request resolving last and overwriting the results of a newer query
- ○ The debounced query being read as a stale closure value inside the then callback
- ○ The fetch request continuing to consume network bandwidth after the effect re-runs
- ○ React throwing an error because setResults was called with a non-array value

Check answer

‹ Previous
Build: Todo App with CRUD

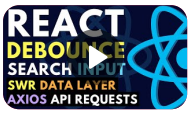
Next ›
Build: Nested Circles (Recursive Component)

LEARN MORE

ARTICLES & DOCS

Debouncing & Throttling explained
css-tricks.com

VIDEOS



Debounced search input + API call
YouTube



Debounce input — stop spamming your API
YouTube



Debounced input with a custom hook
YouTube

Hand-picked references for this lesson.